

Homework 6b

Byron Farmar, Gonzaga MENG 451, Dr. Fitzgerald

6.2

The first thing to change was the mesh. For this problem, a square mesh was chosen.

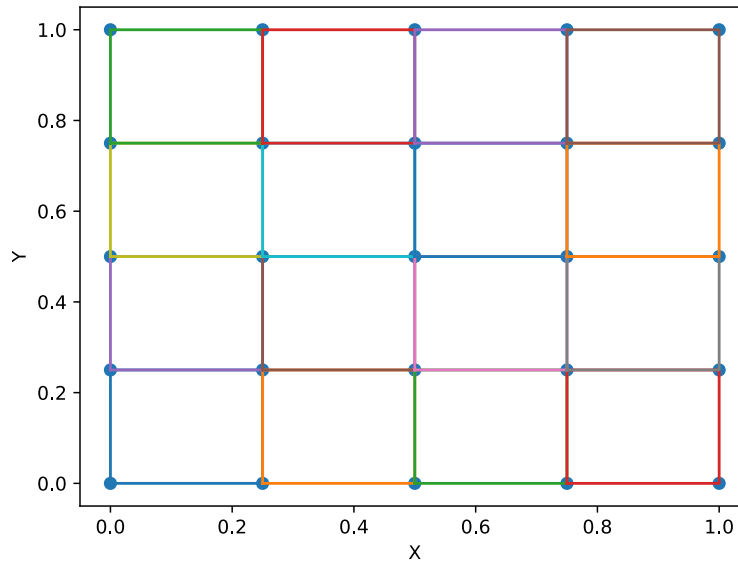


Figure 1: Visualization of a 5x5 mesh.

The boundary conditions were applied and visualized in Figure 2. For the corners, the average of the adjacent boundary values were applied.

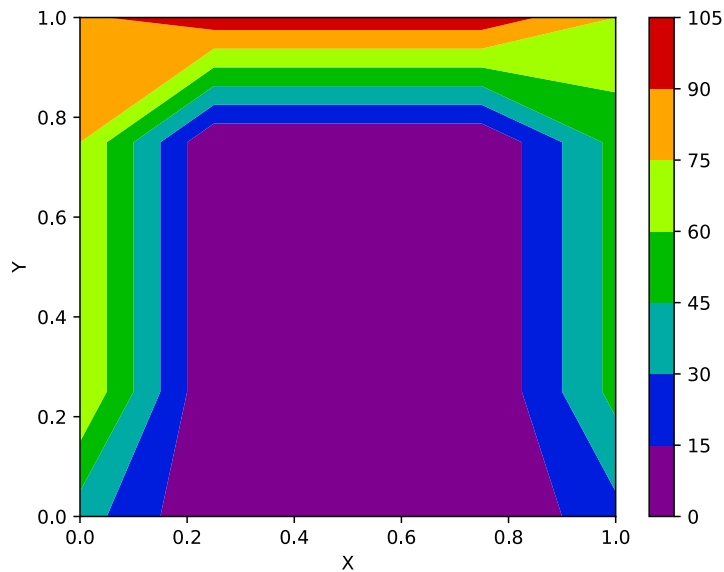


Figure 2: Visualization of Boundary Conditions.

Once the mesh was completed and boundary values applied, the `assemble_stiffness_matrix` function had to be changed so that it passed in `x` and `y` into `element_stiffness_matrix`. `element_stiffness_matrix` had to be modified to follow the heat conduction equation. The solution was then found and plotted in Figure 3.

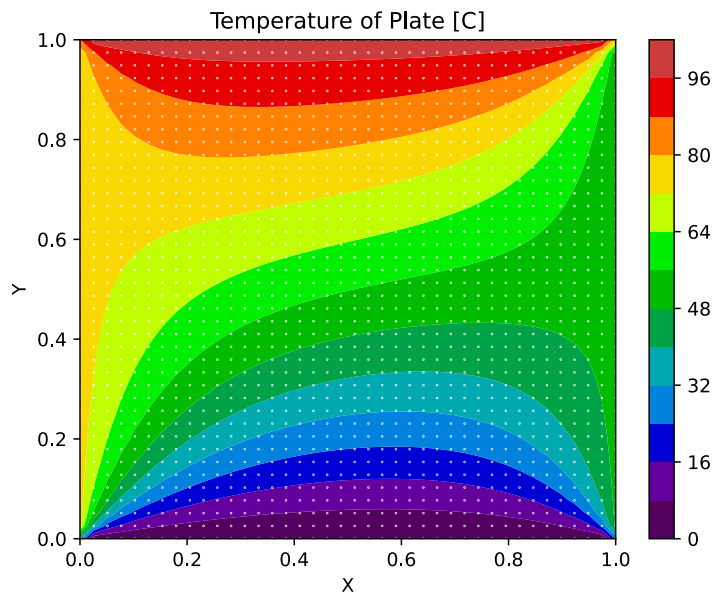


Figure 3: Results from FEM

Compared with the results from Homework 3, as shown in Figure 4, they are super similar. A 40x40 mesh was used to generate Figure 3, and the mesh is overlaid over the contour in white dots.

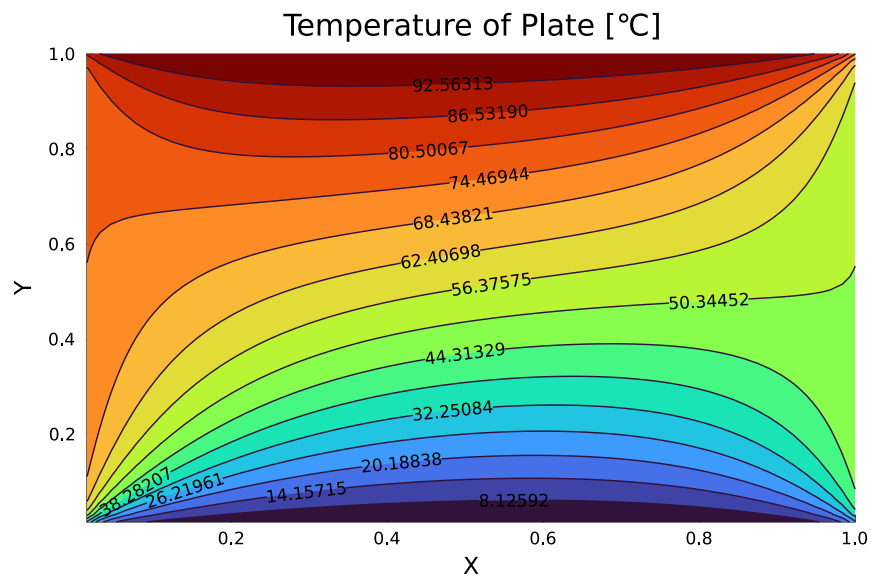


Figure 4: Results from HW3

Code

```

:
:
:
box2D.py
:
import numpy as np
from quadrature import Quadrule
from shapefunctions import shapefunc
from preprocess import std_element_defs # Assuming this exists in Preprocess

def assemble_stiffness(mesh, properties, quad_rules):
    """Assemble the global stiffness matrix."""
    num_dof_per_node = mesh.num_dof_per_node

```

```

totaldofs = num_dof_per_node * mesh.num_nodes
K = np.zeros((totaldofs, totaldofs))

# Loop over each element in the mesh
for element_type, element_connectivity in mesh.element_connectivity.items():
    num_elements = element_connectivity.shape[0]
    num_nodes_in_element = std_element_defs[element_type].num_nodes_in_element
    total_num_dof = num_dof_per_node * num_nodes_in_element
    LM = mesh.LM[element_type]

    for element in range(num_elements):
        # TODO: make sure that point inputs are being gotten correctly
        A = element_connectivity[element, :num_nodes_in_element]
        x, y = mesh.x[A], mesh.y[A]
        # print(A)
        # print("x=", x)
        # print("y=", y)

        ke = element_stiffness(x, y, properties, element_type, quad_rules)

        # Assemble element stiffness into the global stiffness matrix
        for loop1 in range(total_num_dof):
            i = LM[loop1, element]
            for loop2 in range(total_num_dof):
                j = LM[loop2, element]
                K[i, j] += ke[loop1, loop2]

return K

def element_stiffness(
    x_pnts, y_pnts, properties, element_type: str, quad_rules: dict[str, Quadrule]
):
    """Compute the element stiffness matrix for a 1D bar."""
    shapefoo = shapefunc(element_type)

    running_sum = np.zeros((len(x_pnts), len(x_pnts)))
    for xi, wi in quad_rules[element_type].iterator:
        for xj, wj in quad_rules[element_type].iterator:
            # Evaluate the shape function
            _, Nxi, Neta = shapefoo(xi, xj)

            # WARN: check that the shapes for all this is valid
            shape_derivatives = np.array([Nxi, Neta]).T
            # print("shape", shape_derivatives.T)
            # print("Nxi", Nxi)
            # print("Neta", Neta)
            pnt_mat = np.array([x_pnts, y_pnts])
            # print("pnt", pnt_mat)
            # print("x", x_pnts)
            # print("y", y_pnts)

            jacob = pnt_mat @ shape_derivatives
            detJ = np.linalg.det(jacob)

            B = np.linalg.solve(jacob, shape_derivatives.T)

            running_sum += properties.K * B.T @ B * detJ * wi * wj

```

```

return running_sum

def assemble_rhs(mesh, external_forcing, quad_rules):
    """Assemble the global right-hand-side force vector."""
    ned = mesh.num_dof_per_node
    totaldofs = ned * mesh.num_nodes
    F = np.zeros(totaldofs)

    # Loop over each element in the mesh
    for element_type, element_connectivity in mesh.element_connectivity.items():
        num_elements = element_connectivity.shape[0]
        shape_funcs = shapefunc(element_type)
        element_quad_rule = quad_rules[element_type]
        num_nodes_in_element = std_element_defs[element_type].num_nodes_in_element
        total_num_dof = ned * num_nodes_in_element
        LM = mesh.LM[element_type]

        for element in range(num_elements):
            A = element_connectivity[element, :num_nodes_in_element]
            point_inputs_x = mesh.x[A]
            point_inputs_y = mesh.y[A]

            fe = element_forcing(
                point_inputs_x,
                point_inputs_y,
                shape_funcs,
                element_quad_rule,
                external_forcing,
            )

            # Assemble element force into the global force vector
            for loop1 in range(total_num_dof):
                i = LM[loop1, element]
                F[i] += fe[loop1]

    return F

def element_forcing(xe, ye, N, element_quad_rule: Quadrature, external_forcing):
    """Compute the element force vector."""
    ned = 1
    nen = len(xe)
    nee = ned * nen
    fe = np.zeros(nee)

    # Integration loop
    for xi, wi in element_quad_rule.iterator:
        for xj, wj in element_quad_rule.iterator:
            # Evaluate the shape function
            Ne, Nx, Ny = N(xi, xj)

            # Evaluate the external loading at x(xi)
            x = np.dot(Ne, xe)
            y = np.dot(Ne, ye)
            fext = external_forcing(x, y)

```

```

        shape_derivatives = np.array([Nξ, Nη]).reshape(4, -1)
        diff_arr = np.array([xe, ye])

        jacob = diff_arr @ shape_derivatives
        detJ = np.linalg.det(jacob)

        # Integrate
        fe += Ne * fext * detJ * wi * wj

    return fe
:::::::::::::
mainSimpleBar.py
:::::::::::::
import numpy as np
from collections import namedtuple
from preprocess import build_mesh
import quadrature as quad
from box2D import assemble_stiffness, assemble_rhs
from plotting import plot_solution, plot_mesh, plot_bc
import sys

np.set_printoptions(
    threshold=sys.maxsize, linewidth=1000000000, precision=2, suppress=True
)

# Define properties
Properties = namedtuple("Properties", ["w", "h", "K"])
prop = Properties(w=1, h=1, K=1)

def build():
    # Define quad rule
    quad_rules = {
        "quad": quad.gauss_legendre_1d(1),
    }

    # for 3 node quadratic

    one_dimension_size = 40
    one_dim_el = one_dimension_size - 1
    amt_points = one_dimension_size**2
    # make dictionary of elements
    element_connectivity = []
    for i in range(one_dim_el):
        for j in range(one_dim_el):
            element_connectivity.append(
                np.array(
                    [
                        i * one_dimension_size + j,
                        i * one_dimension_size + j + 1,
                        i * one_dimension_size + j + one_dimension_size + 1,
                        i * one_dimension_size + j + one_dimension_size,
                    ],
                    dtype=int,
                )
            )

    element_connectivity = {

```

```

    "quad": np.array(element_connectivity),
}

x = np.linspace(0, prop.w, one_dimension_size)
y = np.linspace(0, prop.h, one_dimension_size)
x, y = np.meshgrid(x, y)

# flag the essential boundary conditions
bc_fix_list = np.zeros((one_dimension_size, one_dimension_size), dtype=int)
# set boundy values to one since they are constrained
bc_fix_list[0, :] = 1
bc_fix_list[one_dimension_size - 1, :] = 1
bc_fix_list[:, 0] = 1
bc_fix_list[:, one_dimension_size - 1] = 1

bc_g_list = np.zeros_like(bc_fix_list, dtype=float)

# set boundy values to one since they are constrained
bc_g_list[0, :] = BoundyConditions.bottom
bc_g_list[one_dimension_size - 1, :] = BoundyConditions.top
bc_g_list[:, 0] = BoundyConditions.left
bc_g_list[:, one_dimension_size - 1] = BoundyConditions.right

# handle corners
bc_g_list[0, one_dimension_size - 1] = (
    BoundyConditions.right + BoundyConditions.bottom
) / 2

bc_g_list[0, 0] = (BoundyConditions.left + BoundyConditions.bottom) / 2

bc_g_list[one_dimension_size - 1, one_dimension_size - 1] = (
    BoundyConditions.right + BoundyConditions.top
) / 2

bc_g_list[one_dimension_size - 1, 0] = (
    BoundyConditions.left + BoundyConditions.top
) / 2

# reshape to be l=d
bc_fix_list = bc_fix_list.reshape((1, amt_points))
bc_g_list = bc_g_list.reshape((1, amt_points))
return (
    x,
    y,
    element_connectivity,
    bc_fix_list,
    bc_g_list,
    quad_rules,
    one_dimension_size,
)

```

```

class BoundyConditions:

```

```

    top = 100
    bottom = 0
    left = 75
    right = 50

```

```

x, y, element_connectivity, bc_fix_list, bc_g_list, quad_rules, one_dimension_size = (
    build()
)

plt = plot_mesh(x, y, element_connectivity["quad"])
# plt.savefig("mesh.svg")

plt = plot_bc(x, y, bc_g_list)
# plt.savefig("boundrys.svg")

mesh = build_mesh(
    x.reshape((-1,)),
    y.reshape((-1,)),
    [],
    element_connectivity,
    1,
    bc_fix_list,
    bc_g_list,
)

def f(x, y):
    return 0

# %% Assemble the global stiffness matrix
K = assemble_stiffness(mesh, prop, quad_rules)

# %% Assemble the global right-hand-side force vector
F = assemble_rhs(mesh, f, quad_rules)

# load the known solutions into the solution matrix
solution = np.zeros(mesh.num_nodes * mesh.num_dof_per_node)
idx = np.where(bc_fix_list == 1)
solution[mesh.ID[idx]] = bc_g_list[idx]

# %% Solve
r1 = mesh.free_range
r2 = mesh.freefix_range
solution[r1] = np.linalg.solve(
    K[np.ix_(r1, r1)], F[r1] - K[np.ix_(r1, r2)] @ solution[r2]
)

def convert_sol_to_two_d(solution):
    # have to unpack to solution into the free range
    twod_sol = bc_g_list.reshape(one_dimension_size, one_dimension_size)
    free_range_vals = iter(solution[r1])
    for i in range(1, one_dimension_size - 1):
        for j in range(1, one_dimension_size - 1):
            twod_sol[i, j] = next(free_range_vals)
    return twod_sol

unpacked_solution = convert_sol_to_two_d(solution)

plt = plot_solution(unpacked_solution, x, y)

```

```

plt.savefig("contour.svg")
:~::~:
plotting.py
:~::~:
import matplotlib.pyplot as plt
import numpy as np

def label_xy():
    plt.xlabel("X")
    plt.ylabel("Y")

def plot_solution(sol, x, y):
    plt.clf()
    label_xy()
    contour = plt.contourf(x, y, sol, levels=15, cmap="nipy_spectral")
    # plt.clabel(contour, colors="black", inline=True, fontsize="8")
    plt.scatter(x, y, color="white", s=0.1)
    plt.colorbar(contour)
    plt.title("Temperature of Plate [C]")
    return plt

def plot_mesh(x, y, connectivity_matrix):
    plt.clf()
    plt.scatter(x, y)
    label_xy()
    x = x.reshape((-1,))
    y = y.reshape((-1,))
    for row in range(connectivity_matrix.shape[0]):
        idx = connectivity_matrix[row, :]
        xe, ye = list(x[idx]), list(y[idx])
        xe.append(xe[0])
        ye.append(ye[0])
        plt.plot(xe, ye)

    return plt

def plot_bc(x, y, bc_list):
    plt.clf()
    label_xy()
    side_size = int(np.sqrt(bc_list.shape[1]))
    bc_list = bc_list.reshape((side_size, side_size))
    contour = plt.contourf(x, y, bc_list, cmap="nipy_spectral")
    plt.colorbar(contour)
    return plt
:~::~:
preprocess.py
:~::~:
import numpy as np

class Mesh:
    def __init__(
        self,
        x,

```



```

        y,
        z,
        IEN,
        ID,
        LM,
        ned,
        neq,
        nnp,
        ng,
        BC_g_list,
        BC_fix_list,
        free_range,
        freefix_range,
    ):
        self.x = x
        self.y = y
        self.z = z
        self.element_connectivity = IEN
        self.ID = ID
        self.LM = LM
        self.num_dof_per_node = ned
        self.num_free_equations = neq
        self.num_nodes = nnp
        self.num_constrained_corrds = ng
        self.BC_g_list = BC_g_list
        self.BC_fix_list = BC_fix_list
        self.free_range = free_range
        self.freefix_range = freefix_range

class ElementDef:
    def __init__(self, name, nen, internal_dim, gmsh_number):
        self.name = name
        self.num_nodes_in_element = nen
        self.internal_dim = internal_dim
        self.gmsh_number = gmsh_number

# Standard GMSH element definitions
std_element_defs = {
    "line": ElementDef("Linear Line", 2, 1, 1),
    "curve": ElementDef("3 node quadratic", 3, 1, 1),
    "triangle": ElementDef("Linear Triangle", 3, 2, 2),
    "quad": ElementDef("4 node quadrilateral", 4, 2, 3),
}

def build_identity_matrix(num_nodes, g_list, dof_per_node, fix_list):
    number_of_free_equations = 0
    count = 0

    # Compute ng
    num_constrained_coords = np.sum(fix_list > 0)

    # Construct ID
    identity_matrix = np.zeros((dof_per_node, num_nodes), dtype=int)
    totaldof = dof_per_node * num_nodes

```

```

if num_constrained_coords > 0:
    count = 0
    for A in range(num_nodes):
        for i in range(dof_per_node):
            if fix_list[i, A] == 1:
                identity_matrix[i, A] = totaldof - count
                count += 1
            else:
                number_of_free_equations += 1
                identity_matrix[i, A] = number_of_free_equations
else:
    for A in range(num_nodes):
        for i in range(dof_per_node):
            number_of_free_equations += 1
            identity_matrix[i, A] = number_of_free_equations

# Fix numbering to be zero-based
identity_matrix = identity_matrix - 1

free_range = range(number_of_free_equations)
freefix_range = range(
    number_of_free_equations, number_of_free_equations + num_constrained_coords
)

return (
    identity_matrix,
    number_of_free_equations,
    num_constrained_coords,
    free_range,
    freefix_range,
)

def build_LM(identity_matrix, IEN):
    ned = identity_matrix.shape[0]
    LM = {}

    for element_type, ien in IEN.items():
        num_elements = ien.shape[0]
        num_nodes_in_element = std_element_defs[element_type].num_nodes_in_element
        lm = np.zeros((num_nodes_in_element * ned, num_elements), dtype=int)

        for e in range(num_elements):
            for a in range(num_nodes_in_element):
                for i in range(ned):
                    p = a + num_nodes_in_element * (i)
                    # print(ien[e, a])
                    # print(identity_matrix)
                    lm[p, e] = identity_matrix[i, ien[e, a]]

        LM[element_type] = lm

    return LM

def build_mesh(x, y, z, element_connectivity, DofPerNode, BC_fix_list, BC_g_list):
    num_nodes = len(x)
    ID, num_free_equations, num_constrained_coords, free_range, freefix_range = (

```

```

        build_identity_matrix(num_nodes, BC_g_list, DofPerNode, BC_fix_list)
    )
    LM = build_LM(ID, element_connectivity)

    return Mesh(
        x,
        y,
        z,
        element_connectivity,
        ID,
        LM,
        DofPerNode,
        num_free_equations,
        num_nodes,
        num_constrained_coords,
        BC_g_list,
        BC_fix_list,
        free_range,
        freefix_range,
    )
):
:-----:
quadrature.py
:-----:
import numpy as np

class Quadrule:
    def __init__(self, label, n, dim, ξ, w):
        self.label = label
        self.n = n
        self.dim = dim
        self.ξ = ξ
        self.w = w
        self.iterator = list(zip(ξ, w)) # Creating an iterable of nodes and weights

def gauss_legendre_1d(n):
    """Computes the 1D Gauss-Legendre quadrature rule."""
    beta = 0.5 / np.sqrt(1 - (2 * np.arange(1.0, n)) ** -2) # Compute beta coefficients
    T = np.diag(beta, -1) + np.diag(beta, 1) # Construct symmetric tridiagonal matrix
    λ, V = np.linalg.eigh(T) # Compute eigenvalues and eigenvectors
    p = np.argsort(λ) # Sort eigenvalues

    ξ = λ[p] # Sorted nodes
    w = 2 * (V[0, p] ** 2) # Compute weights

    return Quadrule("1D GL", n, 1, ξ, w)

:-----:
shapefunctions.py
:-----:
import numpy as np

def line(r):
    """Shape functions for a 2-node line."""
    if isinstance(r, (list, tuple, np.ndarray)):
        r = r[0]

```

```

NN = np.array([(1 - r) / 2, (1 + r) / 2])
Nr = np.array([-1 / 2, 1 / 2])
return NN, Nr

def curve(ξ):
    """
    Shape function for 3 node quadratic
    Make sure x=0 is the middle node "hours later"
    """
    NN = np.array([ξ * (ξ - 1) / 2, 1 - ξ**2, ξ * (ξ + 1) / 2])
    Nξ = np.array([ξ - 1 / 2, -2 * ξ, ξ + 1 / 2])
    return NN, Nξ

def triangle(r, s):
    """Shape functions for a 3-node triangle."""
    NN = np.array([(1 / 2) * (-r - s), (1 / 2) * (1 + r), (1 / 2) * (1 + s)])

    # Derivatives
    Nr = np.array([-1 / 2, 1 / 2, 0])
    Ns = np.array([-1 / 2, 0, 1 / 2])

    return NN, Nr, Ns

def quad(r, s):
    """Shape functions for a 4-node quadrilateral."""
    NN = np.array(
        [
            (1 / 4) * (-1 + r) * (-1 + s),
            (-1 / 4) * (1 + r) * (-1 + s),
            (1 / 4) * (1 + r) * (1 + s),
            (-1 / 4) * (-1 + r) * (1 + s),
        ]
    )

    # Derivatives
    Nr = np.array(
        [(1 / 4) * (-1 + s), (1 / 4) * (1 - s), (1 / 4) * (1 + s), (1 / 4) * (-1 - s)]
    )

    Ns = np.array(
        [(1 / 4) * (-1 + r), (1 / 4) * (-1 - r), (1 / 4) * (1 + r), (1 / 4) * (1 - r)]
    )

    return NN, Nr, Ns

def shapefunc(el_type):
    """Returns the appropriate shape function based on element type."""
    match el_type:
        case "line":
            return line
        case "triangle":
            return triangle
        case "quad":

```

```
        return quad
    case "curve":
        return curve
    case _:
        raise ValueError("Element type not found")
```